gemstone-rs Performance and Safety

Benchmark guidance, GCI loading notes, and threading rules

Generated from the Markdown sources in gemstone-rs/docs.

Contents

Performance and Safety

`gemstone-rs` keeps the low-level GemStone/S GCI boundary explicit. The goal is to make Rust services and tooling fast without hiding session, transaction, or threading rules.

Dynamic GCI Loading

The GCI library is loaded at runtime. Configure it with one of:

```
export GS_LIB=/path/to/GemStone64Bit/lib
export GS_LIB_PATH=/path/to/GemStone64Bit/lib
export GEMSTONE=/path/to/GemStone64Bit
```

The loader already reports which source selected `libgcirpc` in `gemstone-rs doctor`: explicit config, `GS_LIB_PATH`, `GS_LIB`, or `GEMSTONE/lib`, plus the exact path or directory searched. It should keep improving in these directions:

- detect architecture mismatches early
- avoid requiring GemStone headers at Rust build time
- keep all unsafe C ABI calls isolated in `gemstone-gci`

Session Threading

`Session` is deliberately non-`Send` and non-`Sync`. Treat a session as owned by the thread that logged it in.

For web servers:

- use `SessionWorker` when you want one reusable dedicated session lane
- use `SessionWorkerPool` when a service needs a bounded set of reusable lanes
- use `spawn_blocking` or framework blocking helpers for simple per-request probes
- prefer a small `SessionWorkerPool` for web services and reuse

  `gemstone-rs-axum` or `gemstone-rs-actix` instead of copying handler code

- use the adapter `*_from_env` helpers when the HTTP process should start

  before GemStone credentials are configured; `/health/gemstone` will report unavailable until the pool is ready

- keep `x-gemstone-rs-adapter`, `x-gemstone-rs-route`,

`x-gemstone-rs-request-id`, `x-gemstone-rs-request-method`, and `x-gemstone-rs-request-path` headers enabled in local services so route smoke tests and proxy logs can identify health paths and correlate requests

- keep transaction boundaries explicit
- do not share a live session between async tasks

Benchmark Smoke Test

Use the lightweight benchmark smoke script for a coarse local sanity check:

```
ITERATIONS=20 scripts/benchmark_smoke.sh "3 + 4"
```

This measures CLI startup, login, eval, and logout together. It is not a microbenchmark. A future benchmark suite should separate:

- GCI library load time
- login/logout time
- eval/perform latency
- string and array marshalling
- BridgeRoot read/write latency
- codegen wrapper overhead

Rust vs Python

Compare `gemstone-rs` and `gemstone-py` only with equivalent workloads:

- same stone
- same host
- same GemStone user
- same expression or selector
- warm and cold runs reported separately

The first useful comparison is:

```
Rust:   cargo run -p gemstone-rs-cli -- eval "3 + 4"
Python: python -m gemstone_py.cli eval "3 + 4"
```

After that, compare generated-wrapper calls and BridgeRoot mapping operations.

Safety Checklist

- Keep `Session` non-`Send` and non-`Sync`.
- Keep `unsafe` in `gemstone-gci`.
- Prefer typed `Value` / `Oop` conversions over raw integer handling.
- Commit or abort explicitly after writes.
- Bind local tools to `127.0.0.1` by default.
- Keep explorer eval/write operations opt-in.

> This PDF was generated locally from `docs/`. If you edit the Markdown sources, rerun `python docs/build_pdf_docs.py`.